

Pizza Sales Data Analysis – Portfolio Project - SQL

Business Problem

A pizza restaurant collects transactional sales data but lacks clear insights into its performance. The goal of this project is to analyze sales data to answer key business questions such as:

- How much revenue is generated?
- Which products perform best?
- When are peak sales periods?
- What customer purchasing patterns exist?

The objective is to transform raw data into actionable insights using SQL.

1. Environment Setup

- Downloaded and installed **SQL Server Management Studio (SSMS)**.
- Connected SSMS to a local SQL Server instance.
- Created a new database named Pizza DB.

2. Data Cleaning and Preparation (SQL ETL)

- Used **Import Flat File** wizard (right-click database → Tasks → Import Flat File).
- Selected the CSV file pizza_sales.csv.
- Wizard automatically detected data types (e.g., INT, DECIMAL, DATE).

Error Encountered:

Input string was not in a correct format.

The wizard failed because the CSV's date format (DD-MM-YYYY) and certain decimal values were incompatible with the auto-detected SQL Server types.

3. Troubleshooting & Solution

Instead of forcing the wizard, I used a **two-table staging approach**:

Step 3.1 – Import all data as VARCHAR(MAX)

- Re-ran the Import Flat File wizard.
- Manually changed **every column** to VARCHAR(MAX)
- The import succeeded because no type conversion was attempted.

Step 3.2 – Create a clean, correctly typed table

Sql

```
CREATE TABLE dbo.pizza_sales_clean (
    pizza_id INT,
    order_id INT,
    pizza_name_id VARCHAR(50),
    quantity INT,
    order_date DATE,
    order_time TIME,
    unit_price DECIMAL(6,2),
    total_price DECIMAL(8,2),
    pizza_size VARCHAR(50),
    pizza_category VARCHAR(50),
    pizza_ingredients VARCHAR(MAX),
    pizza_name VARCHAR(100)
);
```

Step 3.3 – Convert and insert data safely using TRY CAST / TRY CONVERT

Sql

```
INSERT INTO dbo.pizza_sales_clean
SELECT
    TRY_CAST(pizza_id AS INT) AS pizza_id,
    TRY_CAST(order_id AS INT) AS order_id,
    pizza_name_id,
    TRY_CAST(quantity AS INT) AS quantity,
    TRY_CONVERT(DATE, order_date, 105) AS order_date, -- 105 = dd-mm-yyyy
    TRY_CONVERT(TIME, order_time) AS order_time,
    TRY_CAST(unit_price AS DECIMAL(6,2)) AS unit_price,
    TRY_CAST(total_price AS DECIMAL(8,2)) AS total_price,
    pizza_size,
    pizza_category,
    pizza_ingredients,
    pizza_name
FROM dbo.pizza_sales; -- source table (all varchar max)
```

Step 3.4 – Validate and replace the original table

Checked for any conversion failures (none were found).

```
SELECT *
FROM dbo.pizza_sales_clean
WHERE pizza_id IS NULL
    OR order_id IS NULL
    OR quantity IS NULL
    OR order_date IS NULL
    OR order_time IS NULL
    OR unit_price IS NULL
    OR total_price IS NULL;
```

```
SELECT order_date, pizza_id
FROM dbo.pizza_sales
WHERE TRY_CONVERT(DATE, order_date, 105) IS NULL
    AND order_date IS NOT NULL;
```

Step 3.5 – Final Table setup

- Renamed the raw table to pizza_sales_raw for audit/backup.
- Renamed the clean table to pizza_sales to allow existing queries to work seamlessly.

Sql

- -- Step 1: Rename the raw table
- EXEC sp_rename 'dbo.pizza_sales', 'pizza_sales_raw';
- -- Step 2: Rename your clean table to the original name
- EXEC sp_rename 'dbo.pizza_sales_clean', 'pizza_sales';

Data Cleaning Summary (SQL-based):

- Converted string dates (DD-MM-YYYY) to DATE using TRY_CONVERT with format 105
- Cast VARCHAR columns to INT, DECIMAL, TIME using TRY_CAST
- Validated for conversion failures with NULL checks (no missing values found)
- Preserved original raw data for audit

4. Final Data Structure

After cleaning, the pizza_sales table has proper data types:

Column	Type	Example
pizza_id	INT	1
order_id	INT	1
pizza_name_id	VARCHAR(50)	3awaiian_m
quantity	INT	1
order_date	DATE	2015-01-01
order_time	TIME	11:38:36
unit_price	DECIMAL(6,2)	13.25
total_price	DECIMAL(8,2)	13.25
pizza_size	VARCHAR (10)	M
pizza_category	VARCHAR(50)	Classic
pizza_ingredients	VARCHAR(MAX)	Sliced Ham, ...
pizza_name	VARCHAR(100)	The Hawaiian Pizza

Data model note: This table contains one row per line item (each pizza in an order). Multiple rows share the same order_id when an order contains multiple pizzas.

5. Exploratory Data Analysis (SQL)

With a clean, performance-optimized table, I can now run sql queries to gain business insights

5. 1: KPI & Dashboard Metrics

1. Total Revenue: What is the total amount of money made from all pizza sales?

```
select sum(total_price) as Revenue from pizza_sales;
```

	Revenue
1	817860.05

Insight: Identifies which pizza categories contribute most to revenue, helping prioritize high-performing products.

5. Total Pizzas Sold - How many individual pizzas were sold in total?

```
select sum(quantity) as total_pizza_sold from pizza_sales
```

Results		Messages	
	Total_pizza_sold		
1	49574		

6. Total Orders

How many unique transactions took place?

```
select count(distinct(order_id)) as total_orders from pizza_sales
```

Results

Messages

	Total_Orders
1	21350

Insight:

These metrics provide a high-level overview of business performance and are used to power the Excel dashboard.

5. 2 Revenue Analysis

2. Total Revenue by pizza category?

Which pizza category (Classic, Chicken, Supreme, Veggie) brings in the most money?

```
select
    pizza_category,
    sum(total_price) as category_revenue
from pizza_sales
group by pizza_category
order by category_revenue DESC
```

Results		Messages
	pizza_category	category_revenue
1	Classic	220053.10
2	Supreme	208197.00
3	Chicken	195919.50
4	Veggie	193690.45

Insight: Identifies which pizza categories contribute most to revenue, helping prioritize high-performing products.

3.Total revenue by month

How does revenue change month by month?

```
select
    year(order_date) as order_year,
    month(order_date) as order_month,
    sum(total_price) as monthly_revenue
from pizza_sales
group by year(order_date), month(order_date)
order by order_year, order_month
```

	order_year	order_month	monthly_revenue
1	2015	1	69793.30
2	2015	2	65159.60
3	2015	3	70397.10

Insight: Highlights seasonal trends and helps forecast demand.

10. revenue by pizza size

Which pizza size (s,m,l, etc) generates the most revenue

```
select pizza_size, sum(total_price) as size_revenue
from pizza_sales
group by pizza_size
order by size_revenue desc
```

	pizza_size	size_revenue
1	L	375318.70
2	M	249382.25
3	S	178076.50

17. % of sales by pizza category

```
select
    pizza_category,
    sum(total_price) as total_revenue,
    cast(sum(total_price) * 100.0 / (select sum(total_price) from pizza_sales) as decimal(10,2))
as percentage_of_total_sales
from pizza_sales
group by pizza_category
```

order by percentage_of_total_sales desc

	pizza_category	total_revenue	percentage_of_total_sales
2	Supreme	208197.00	25.46
3	Chicken	195919.50	23.96
4	Veggie	193690.45	23.68

12. bottom 3 pizzas by revenue

Which five pizzas generate the most money, not just quantity sold

select top 3 pizza_name, sum(total_price) as revenue — return top 3 after asc sorting so it will show the lowest

from pizza_sales
group by pizza_name
order by revenue asc

	pizza_name	revenue
1	The Brie Carre Pizza	11588.50
2	The Green Garden Pizza	13955.75
3	The Spinach Supreme Pizza	15277.75

Insight:

Revenue is driven by specific categories and sizes, while some pizzas contribute significantly less, highlighting opportunities for optimization.

5. 3: Product Performance

8 Best-Selling Pizza (by quantity) - Which pizza sold the most units

select top 1
pizza_name,
sum(quantity) as total_sold
from pizza_sales
group by pizza_name
order by total_sold DESC

	pizza_name	total_sold
1	The Classic Deluxe Pizza	2453

19. top 5 best sellers (by total pizza sold)

select top 5
pizza_name,
sum(quantity) as total_pizza_sold
from pizza_sales
group by pizza_name
order by total_pizza_sold desc

	pizza_name	total_pizza_sold
1	The Classic Deluxe Pizza	2453
2	The Barbecue Chicken Pizza	2432
3	The Hawaiian Pizza	2422
4	The Pepperoni Pizza	2418
5	The Thai Chicken Pizza	2371

20. bottom 5 worst selling pizza (by number of pizzas sold)

```
Select top 5 pizza_name, sum(quantity) as Total_Pizza_Sold
From pizza_sales
Group by pizza_name
Order by Total_Pizza_Sold asc
```

Results Messages		
	pizza_name	Total_Pizza_Sold
1	The Brie Carre Pizza	490
2	The Mediterranean Pizza	934
3	The Calabrese Pizza	937
4	The Spinach Supreme Pizza	950
5	The Soppressata Pizza	961

11. most popular pizza by size and category

Within each pizza category, which size sells the best

```
select pizza_category, pizza_size, sum(quantity) as qty_sold
from pizza_sales
group by pizza_category, pizza_size
order by pizza_category, qty_sold --sort by category first, then within each category sort by qty sold
```

	pizza_category	pizza_size	qty_sold
1	Chicken	S	2224
2	Chicken	M	3894
3	Chicken	L	4932
4	Classic	XXL	28

18. total pizzas sold by category in Feb 2015

```
select
    pizza_category,
    sum(quantity) as total_pizzas_sold
from pizza_sales
where Month(order_date) = 2 and year(order_date) = 2015
group by pizza_category
order by total_pizzas_sold desc
```

	pizza_category	total_pizzas_sold
1	Classic	1178
2	Supreme	964
3	Veggie	944
4	Chicken	875

Insight:

A small number of pizzas dominate sales, while others underperform, suggesting potential menu improvements.

5. 4: Time-based Analysis

9. PEAK order hours

During which hours of the day do customers place the most orders?

```
select
    datepart(hour, order_time) as hour_of_day,
    count(distinct(order_id)) as order_count
from pizza_sales
group by datepart(hour, order_time)
order by order_count desc
```

	hour_of_day	order_count
1	12	2520
2	13	2455
3	18	2399

Insight: Identifies busiest hours (e.g., lunch/dinner), useful for staffing decisions.

13. Daily sales trend (revenue by day of week)

Which day of the week does the company makes the most money (busiest day?)

```
select
    DATENAME(weekday, order_date) as day_name,
    sum(total_price) as daily_revenue
from pizza_sales
group by DATENAME(weekday, order_date)
order by daily_revenue desc
```

	day_name	daily_revenue
1	Friday	136073.90
2	Thursday	123528.50
3	Saturday	123182.40
4	Wednesday	114408.40
5	Tuesday	114133.80

Insight: Shows which days generate the most revenue.

14. daily trends for total orders

```
select
    datename(weekday, order_date) as day_of_week,
    count(distinct order_id) as total_orders
from pizza_sales
group by datename(weekday, order_date)
order by total_orders desc
```

	day_of_week	total_orders
2	Thursday	3239
3	Saturday	3158
4	Wednesday	3024
5	Tuesday	2973
6	Monday	2794
7	Sunday	2624

Insight: Shows day of the week is the busiest

18. pizza popularity by hour (lunch vs dinner)

Which pizzas sell best during lunch hours (12:00–14:00) versus dinner hours (17:00–19:00)

```
select
    case
        when datepart(hour, order_time) between 12 and 14 then 'Lunch'
        when datepart(hour, order_time) between 17 and 19 then 'Dinner'
        else 'other'
    end as meal_period,
    pizza_name,
    sum(quantity) as total_sold
from pizza_sales
where datepart(hour, order_time) between 12 and 14
    or datepart(hour, order_time) between 17 and 19
group by
    case
        when datepart(hour, order_time) between 12 and 14 then 'Lunch'
        when datepart(hour, order_time) between 17 and 19 then 'Dinner'
        else 'other'
    end,
    pizza_name
order by meal_period, total_sold desc;
```

	meal_period	pizza_name	total_sold
1	Dinner	The Barbecue Chicken Pizza	767
2	Dinner	The Pepperoni Pizza	734
3	Dinner	The Classic Deluxe Pizza	725
4	Dinner	The Hawaiian Pizza	724
5	Dinner	The California Chicken Pizza	717

Insight:

Sales peak during lunch and dinner hours, indicating predictable demand cycles that can guide staffing and promotions.

5. 4: Customer Behaviour

4. Average Order Value

On average, how much money does a customer spend per order?

```
select
    sum(total_price) / count(distinct(order_id)) as avg_order_value
from pizza_sales
```

	avg_order_value
1	38.307262

Insight: Indicates how much customers typically spend per order.

7. Average Pizzas Per Order

Out of all orders, how many pizzas does a typical customer buy in a single transaction?

= total pizza sold/total number of orders

```
select
    sum(quantity) as total_pizzas,
    count(distinct(order_id)) as total_orders,
    sum(quantity) * 1.0 / count(distinct(order_id)) as avg_pizzas_per_order
from pizza_sales
```

	total_pizzas	total_orders	avg_pizzas_per_order
1	49574	21350	2.321967213114

Insight: Provides understanding of customer purchasing patterns.

5. 5: Advanced Analysis – Order-Level Analysis

15. high-value orders

Which orders had the highest total spending?

```
select top 5 order_id, sum(total_price) as order_total
from pizza_sales
group by order_id
order by order_total desc
```

	order_id	order_total
1	18845	444.20
2	10760	417.15
3	1096	285.15
4	6169	284.00
5	740	280.95

16. What did the largest order contain?

```
SELECT pizza_name, quantity, total_price
FROM pizza_sales
WHERE order_id = 18845;
```

	pizza_name	quantity	total_price
1	The Barbecue Chicken Pizza	3	50.25
2	The Big Meat Pizza	2	24.00
3	The Calabrese Pizza	1	16.25
4	The Chicken Alfredo Pizza	1	16.75
5	The Chicken Pesto Pizza	1	12.75

Insight:

High-value orders suggest bulk purchasing behavior, likely from group or event-based customers.

5. 6: Data Quality & Distribution – we clean data before we can query it

16. Missing Values Check (Data Quality)

Are there any missing values

- Identifies incomplete data (e.g., missing order_date, total_price, pizza_name).
- Helps decide whether to clean, impute, or discard affected rows.

```
SELECT
    COUNT(CASE WHEN order_id IS NULL THEN 1 END) AS missing_order_id,
    COUNT(CASE WHEN order_date IS NULL THEN 1 END) AS missing_order_date,
    COUNT(CASE WHEN order_time IS NULL THEN 1 END) AS missing_order_time,
    COUNT(CASE WHEN pizza_name IS NULL THEN 1 END) AS missing_pizza_name,
    COUNT(CASE WHEN quantity IS NULL THEN 1 END) AS missing_quantity,
    COUNT(CASE WHEN total_price IS NULL THEN 1 END) AS missing_total_price
```

FROM pizza_sales;

	missing_order_id	missing_order_date	missing_order_time	missing_pizza_name	missing_quantity	missing_total_price
1	0	0	0	0	0	0

17. order size distribution

How many orders had 1 pizza, 2 pizza, 3 pizzas etc?

```
SELECT order_size, COUNT(*) AS number_of_orders
FROM (
    SELECT order_id, SUM(quantity) AS order_size
    FROM pizza_sales
    GROUP BY order_id
) AS order_totals
GROUP BY order_size
ORDER BY order_size;
```

	order_size	number_of_orders
1	1	8111
2	2	6152
3	3	3207

6. Skills Demonstrated

- SQL Server / SSMS environment setup
- Handling real-world data import errors (date format mismatches)
- Staging table pattern for safe ETL
- Data type conversion with TRY_CAST and TRY_CONVERT (including format 105 for dates)
- Data validation and NULL checking
- Table renaming for audit trails
- Aggregate functions (SUM, COUNT, AVG) with GROUP BY
- Date/time extraction (YEAR, MONTH, DATEPART, DATENAME)
- Conditional logic with CASE
- Subqueries (order size distribution)
- Business KPI calculation (average order value, pizzas per order)
- Missing value detection

Conclusion

This project showcases an end-to-end data analysis workflow in SQL: from cleaning a messy CSV, to structuring a production-ready table, to answering real business questions with SQL queries. The approach is reproducible and demonstrates professional data hygiene.